

Lab 3: Layered Architecture Implementation (CRUD Operations)

This lab focuses on the practical implementation of the Leave Management feature using the Layered Architecture designed in Lab 2. We will set up the project directory structure and code the Create, Read, Update, and Delete (CRUD) operations for a Leave Request, thereby demonstrating the strict flow of data control between layers.

Objectives

- 1. Set up the project structure with clearly separated directories (packages/modules) for the 3 architectural layers (Presentation, Business Logic, Persistence).
- 2. Implement the LeaveRequest object model (Entity) and core CRUD operations.
- 3. Ensure a strict dependency flow: Controller $\rightarrow$ Service $\rightarrow$ Repository.

Tool & Technology Setup

We will use Node.js with the Express.js framework (matching your project's Tech Stack) and an in-memory array to simulate the database in the Persistence layer.

Tool	Purpose	Installation/Setup Guide
Node.js	Runtime environment.	Download and install Node.js from the official website.
Express.js	Lightweight web framework for building the Presentation Layer (Controller/API).	Open the terminal and run: npm install express
IDE/Code Editor	Writing and running code (e.g., VS Code).	Use your existing VS Code.
Postman / cURL	API testing tools to verify the operational flow.	Install the Postman application or use the Thunder Client extension in VS Code.

Activity Practice 1: Project Setup and Model Definition

Goal: Create the basic folder structure and define the data model.

Step-by-Step Instructions & Coding Guide:

1. **Create Project Directory:** Open your Terminal and run the following commands:

```
bash
```

Show full code block

```
mkdir hrm_leave_layered
```

```
cd hrm_leave_layered
```

```
# Initialize Node.js project
```

```
npm init -y
```

```
# Install Express
```

```
npm install express
```

2. **Create Layer Folders/Modules:** Create subdirectories representing the layers and the core application files.

```
bash
```

```
mkdir presentation business_logic persistence models
```

```
touch app.js
```

3. **Define the Model Class:** This class defines the basic structure of the data object that will be passed between layers. Create file: models/LeaveRequest.js

```
javascript
```

Show full code block

```
// Defines the core data structure used throughout the application
```

```
class LeaveRequest {
```

```
  constructor(id, user_id, reason, total_days) {
```

```
    this.id = id;
```

```
    this.user_id = user_id;
```

```
    this.reason = reason;
```

```
    this.total_days = total_days;
```

```
    this.status = 'PENDING';
```

```
    this.created_at = new Date();
```

```
  }
```

```

toJSON() {
  // Helper function for JSON conversion in the Presentation Layer
  return {
    id: this.id,
    user_id: this.user_id,
    reason: this.reason,
    total_days: this.total_days,
    status: this.status,
    created_at: this.created_at
  };
}
}

module.exports = LeaveRequest;

```

Activity Practice 2: Data Access Layer (Persistence Layer / Repository)

Goal: Implement the component that interacts directly with the data store (simulated by an in-memory map/array).

Step-by-Step Instructions & Coding Guide:

1. **Implement the Repository:** Create a class to handle data access. This layer is completely unaware of HTTP Requests or Business rules. Create file: persistence/leaveRepository.js

javascript

Show full code block

```
const LeaveRequest = require('../models/LeaveRequest');
```

```
// Simulate Database (Data Layer)

const leave_db = new Map();

let next_id = 1;

class LeaveRepository {

  /** Layer 3: Solely responsible for direct CRUD operations with the data store */

  create(user_id, reason, total_days) {

    const id = next_id.toString();

    const newLeave = new LeaveRequest(id, user_id, reason, total_days);

    leave_db.set(id, newLeave);

    next_id++;

    return newLeave;

  }

  findAll() {

    // Returns a list of all LeaveRequest objects

    return Array.from(leave_db.values());

  }

  findById(id) {

    return leave_db.get(id);

  }

  updateStatus(id, newStatus) {

    if (leave_db.has(id)) {

      const leave = leave_db.get(id);
```

```

        leave.status = newStatus;
        return leave;
    }
    return null;
}

delete(id) {
    if (leave_db.has(id)) {
        leave_db.delete(id);
        return true;
    }
    return false;
}
}

module.exports = LeaveRepository;

```

Activity Practice 3: Business Logic Layer (Service)

Goal: Implement the component that encapsulates company business rules and orchestrates calls to the Persistence Layer.

Step-by-Step Instructions & Coding Guide:

1. **Implement the Service:** Create a class providing core business functions, using the Repository for data access. Create file: `business_logic/leaveService.js`

javascript

Show full code block

```

const LeaveRepository = require('../persistence/leaveRepository');

// The Service layer requires (needs) the Repository layer

```

```
class LeaveService {  
    /** Layer 2: Handles business rules and data validation. */  
  
    constructor() {  
        // Instantiate the Repository to use its methods  
        this.repo = new LeaveRepository();  
    }  
  
    createLeaveRequest(user_id, reason, total_days) {  
        // Business Rule 1: Must have a reason and a valid number of days  
        if (!reason || total_days <= 0) {  
            throw new Error("Invalid data: Reason is required and total days must be greater  
than 0.");  
        }  
  
        // Business Rule 2 (Based on Project Report): Block consecutive leaves > 3 days  
        if (total_days > 3) {  
            throw new Error("Cannot take more than 3 consecutive days off to ensure  
operational shifts!");  
        }  
  
        // Delegate to the Persistence Layer (Strict downward flow)  
        return this.repo.create(user_id, reason, total_days);  
    }  
  
    getAllLeaves() {  
        return this.repo.findAll();  
    }  
}
```

```

    }

    getLeave(id) {
      const leave = this.repo.findById(id);
      if (!leave) {
        throw new Error(`Leave request with ID not found: ${id}`);
      }
      return leave;
    }

    approveLeave(id) {
      const leave = this.getLeave(id); // Can throw an Error if not found
      if (leave.status !== 'PENDING') {
        throw new Error("Only PENDING requests can be approved.");
      }
      return this.repo.updateStatus(id, 'APPROVED');
    }
  }
}

module.exports = LeaveService;

```

Activity Practice 4: Presentation Layer (Controller / API)

Goal: Implement the component that receives HTTP Requests, calls the Service Layer for processing, and formats the response returned to the Client.

Step-by-Step Instructions & Coding Guide:

1. **Implement the Controller (Express App):** This layer requires the Service Layer.
Create/Edit file: app.js

javascript

Show full code block

```
const express = require('express');
const LeaveService = require('./business_logic/leaveService');
// The Controller/Router layer requires (needs) the Service layer

const app = express();
app.use(express.json());

const leaveService = new LeaveService();

// --- Endpoints (Controllers) ---

app.post('/api/leaves', (req, res) => {
  // Layer 1: Receive input from Request and delegate to Layer 2
  const { user_id, reason, total_days } = req.body;
  try {
    const leave = leaveService.createLeaveRequest(user_id, reason, total_days);
    // Layer 1: Format the Response for the client
    res.status(201).json(leave.toJSON());
  } catch (error) {
    // Layer 1: Catch exceptions thrown by Layer 2 and convert to HTTP 400
    res.status(400).json({ error: error.message });
  }
});

app.get('/api/leaves', (req, res) => {
```

```

// Layer 1: Delegate query to Layer 2
const leaves = leaveService.getAllLeaves();

// Layer 1: Convert array of Objects to JSON array
res.status(200).json(leaves.map(l => l.toJSON()));
});

app.put('/api/leaves/:id/approve', (req, res) => {
  try {
    const updatedLeave = leaveService.approveLeave(req.params.id);
    res.status(200).json({ message: "Request approved!", data:
updatedLeave.toJSON() });
  } catch (error) {
    // Return 404 if not found, or 400 if it's a business error
    const statusCode = error.message.includes("not found") ? 404 : 400;
    res.status(statusCode).json({ error: error.message });
  }
});

// Start the server
const PORT = 5000;
app.listen(PORT, () => {
  console.log(`Leave Management Layer server is running at
http://localhost:${PORT}`);
});

```

2. Run and Test:

- Action: Run the application from your Terminal: `node app.js`
- Action: Use Postman or cURL to test the Endpoints:

- **Create Request (POST):** Call POST `http://127.0.0.1:5000/api/leaves` with JSON body: `{"user_id": 4, "reason": "Sick leave", "total_days": 2}`
- **Test Business Rule (POST):** Attempt to create a request with `total_days: 5` to see how Layer 2 blocks it and throws a 400 error.
- **Get List (GET):** Call GET `http://127.0.0.1:5000/api/leaves`
- **Approve Request (PUT):** Call PUT `http://127.0.0.1:5000/api/leaves/1/approve`